خطة المراجعة — للمعلم

حلّ مشكلات المدرسة بايثون: اختبار الوحدة

الحلول المثلى، المستوى ٣، اختبار الوحدة

المدة
٣٥ دقيقة

المواد
سبّورة، طباشير، ورق. لا حاجة إلى أجهزة.

أهداف التعلّم

- جدول حصص يعمل
- امتحانات بلا تعارض
- طريق حافلة المدرسة
- توزيع المعلّمين على الفصول

النشاط: النمذجة في فرق

١. التمهيد (٥ دقائق): لكل درس من الدروس الأربعة، يسمّي الفصل قرارًا حقيقيًا في مدرسته: جدولًا، أو أسبوع امتحانات، أو طريقًا، أو توزيع مهام.
٢. النمذجة (١٥ دقيقة): يأخذ كل فريق قرارًا ويكتب على الورق خياراته (المتغيّرات) وقواعده (القيود) وهدفه، بأوضح ما يستطيع، مع سطر بايثون حيث أمكن.
٣. التبادل (١٠ دقائق): تتبادل الفرق الأوراق. ويبحث كل فريق عن قاعدة ناقصة واحدة، من النوع الذي لن يخمّنه برنامج الحل أبدًا، ويكتبها.
٤. الختام (٥ دقائق): اسأل عن كل نموذج: على مَن قد تكون أفضل خطة أقسى، وما القاعدة أو الهدف الثاني الذي يحميه؟

لا حاجة إلى أجهزة. وإن كان معك هاتف، فشغّل نموذج أحد الفرق في مختبر «صِغ النموذج بـ CP-SAT» في النهاية.

للنقاش

- أيّ قرار مدرسي ستنمذجه أولًا، ولماذا؟
- متى تكون القاعدة السريعة كافية؟
- مَن يجب أن يرى قواعد النموذج قبل أن يقرّر؟

بطاقة الخروج

- قرار مدرسي سأنمذجه، وهدفه، وقاعدة عدالة واحدة له.

جرّبها في المدرسة غدًا

اختر قرارًا واحدًا في مدرستك هذا الفصل الدراسي (المناوبات، أو مواعيد الامتحانات، أو الحافلة، أو مَن يدرّس ماذا). واكتب مع زميلين خياراته وقواعده وهدفه، وصِغ أصغر صورة منه في مختبر «صِغ النموذج بـ CP-SAT»، واعرض النتيجة على من تمسّهم.

ورقة الاختبار — لكل فريق

حلّ مشكلات المدرسة ببايثون: اختبار الوحدة

الحلول المثلى، المستوى ٣، اختبار الوحدة

الفريق _____ التاريخ _____

١ **في نموذج جدول بأداة CP-SAT، ما متغيّر كل درس؟**

○ الحصة التي يوضع فيها ○ عدد طلاب الفصل ○ اسم معلمه

٢ **تعني INFEASIBLE أن برنامج الحل نفد وقته قبل أن يجد جدولًا.**

○ صح ○ خطأ

٣ **يجب أن تمنع القاعدة الثانية المعلم (واحد لكل مادة) من أن يكون في فصلين في وقت واحد، لكن الجداول ما زالت متعارضة. أيّ سطر فيه الخطأ؟**

ضع دائرة حول السطر الذي فيه الخطأ، ثم اكتبه صحيحًا:

```
1  for c in classes:
2      m.add_all_different([p[c, s] for s in subjects])
3  for s in subjects:
4      m.add_all_different([p[c, s] for s in subjects])
```

٤ **صِل كل مشكلة مدرسية بنوع المسألة التي تمثّلها.**

مواعيد امتحانات بلا تعارض	• • • •	البائع المتجوّل
الحافلة التي تزور كل القرى	• • • •	قاعدة «كلها مختلفة»
معلم واحد لكل فصل	• • • •	تلوين المخطّط
لا فصل بدرسين في وقت واحد	• • • •	التوزيع

٥ **أيّ مما يلي يثبت أن ٣ مواعيد امتحان هي الأقل الممكن؟ اختر كل ما ينطبق.**

☐ يقول برنامج الحل OPTIMAL عند تصغير عدد المواعيد ☐ وجد DSatur خطة بثلاثة مواعيد

☐ ثلاثة امتحانات يتعارض كلٌّ منها مع الباقي ☐ استعمل التلوين الجشع ٤ مواعيد

٦ **رتّب: بناء طريق الحافلة وتحسينه.**

☐ احتفظ بالتغيير إذا قصر الطريق

☐ كرّر حتى لا يفيد أي تبديل

☐ ابنِ طريقًا بقاعدة «أقرب جار»

☐ اعكس المقطع الذي بينهما

☐ أزِل طريقين

٧ ماذا يطبع هذا البرنامج؟

```
d = [[0, 2, 4], [2, 0, 3], [4, 3, 0]]
route = [0, 1, 2]
back = route[1:] + [0]
print(sum(d[a][b] for a, b in zip(route, back)))
```

○ 6 ○ 5 ○ 9

٨ أكمل العادة.

احكم على الخطة مرتين: من خلال _____ ، ومن خلال الحالة _____ ، واتفق على

_____ مع من تمسّهم.

بنك الكلمات: القاعدة، الحصة، الأسوأ، المجموع، اللون

٩ أكمل نموذج التوزيع: فصل واحد بالضبط لكل معلم، وأفضل مجموع للدرجات.

```
for t in range(n):
    m. _____ (x[t])
for c in range(n):
    m.add_exactly_one([x[t][c] for t in range(n)])
m. _____ (sum(score[t][c] * x[t][c]
    for t in range(n) for c in range(n)))
```

بنك الكلمات: `maximize` `minimize` `add_exactly_one` `add_all_different`

١٠ رتّب أسطر قاعدة «أقرب جار».

رقّم الأسطر بالترتيب (١، ٢، ٣...)، وارسم أمام السطر سهمًا ← عن كل إزاحة يحتاج إليها. اشطب أي سطر لا ينتمي إلى البرنامج.

- [ ] `while len(route) < n:`
- [ ] `route = [0]`
- [ ] `route.append(max(left, key=lambda v: last[v]))`
- [ ] `route.append(min(left, key=lambda v: last[v]))`
- [ ] `left = [v for v in range(n) if v not in route]`
- [ ] `last = d[route[-1]]`

الإجابات — للمعلم

حلّ مشكلات المدرسة بايثون: اختبار الوحدة

الحلول المثلى، المستوى ٣، اختبار الوحدة

١ **الحصة التي يوضع فيها**

يقرّر برنامج الحل متى يكون كل درس، فالحصة هي المتغيّر.

٢ **خطأ**

INFEASIBLE إثبات أنه لا يوجد جدول يلتزم بكل القواعد.

٣ **السطر 4:** `m.add_all_different([p[c, s] for c in classes])`

لكل مادة يجب أن تشمل القاعدة دروس تلك المادة في كل الفصول: مُرّ على الفصول.

٤ **مواعيد امتحانات بلا تعارض — تلوين المخطّط**
الحافلة التي تزور كل القرى — البائع المتجوّل
معلم واحد لكل فصل — التوزيع
لا فصل بدرسين في وقت واحد — قاعدة «كلها مختلفة»

تسمية نوع المسألة تخبرك بالطرق والمختبرات المفيدة: التلوين للتعارضات، والطرق للحافلة، والتوزيع للمزاوجة، وقواعد «كلها مختلفة» للجداول.

٥ **ثلاثة امتحانات يتعارض كلٌّ منها مع الباقي، يقول برنامج الحل OPTIMAL عند تصغير عدد المواعيد**

الزمرة المكوّنة من ثلاثة امتحانات تحتاج إلى ثلاثة مواعيد في أي خطة، وOPTIMAL تعني أن برنامج الحل أثبت أن الأقل لا ينجح.

٦ **من الأعلى إلى الأسفل: ٤، ٥، ١، ٣، ٢**

تبني القاعدة السريعة طريقًا كاملًا أولًا، ثم تحسّنه 2-opt تبديلًا بعد تبديل، وتتوقّف حين لا يفيد أي تبديل واحد، وقد يكون ذلك قبل الأفضل.

٧ **9**

يقرن zip كل محطة بالتي تليها، وتنتهي back بالمدرسة: من 0 إلى 1 (٢)، ومن 1 إلى 2 (٣)، ومن 2 إلى 0 (٤)، والمجموع ٩.

٨ **احكم على الخطة مرتين: من خلال المجموع، ومن خلال الحالة الأسوأ، واتفق على القاعدة مع من تمسّهم.**

قد تكون الخطة صاحبة أفضل مجموع قاسية على شخص واحد: أطول رحلة في الحافلة، أو ثلاثة امتحانات متتالية، أو أسوأ توزيع.

٩

```
for t in range(n):
    m.add_exactly_one(x[t])
for c in range(n):
    m.add_exactly_one([x[t][c] for t in range(n)])
m.maximize(sum(score[t][c] * x[t][c]
    for t in range(n) for c in range(n)))
```

يحتاج كل صف من متغيّرات نعم أو لا إلى «نعم» واحدة بالضبط، مثل كل عمود، والهدف أكبر مجموع للدرجات، فاستعمل maximize.

١٠ **من الأعلى إلى الأسفل: ٢، ١، ✗، ٥، ٤، ٣**

```
route = [0]
while len(route) < n:
    last = d[route[-1]]
    left = [v for v in range(n) if v not in route]
    route.append(min(left, key=lambda v: last[v]))
```

ابدأ من المدرسة، ثم ما دامت هناك أماكن باقية، انظر إلى المسافات من آخر محطة وأضف أقرب مكان لم يُزَر: min لا max.